

Privacy-Preserving Real-Time Crowd Counting on IoT enabled Edge Devices Using YOLOv8n and Norfair

W.T.D. Thilini Hansika

Dept. of Electrical and Telecom. Eng.
South Eastern University
Olivil, Sri Lanka
thilinih20h@gmail.com

M.A. Pramod Madhawa

Dept. of Electrical and Telecom. Eng.
South Eastern University
Olivil, Sri Lanka
pramodmadawa95@gmail.com

N.T.M. Sajith*

Dept. of Computer Science and Eng.
South Eastern University
Olivil, Sri Lanka
ntmsajith@seu.ac.lk

M.N.M. Aashiq*

Dept. of Computer Science and Eng.
South Eastern University
Olivil, Sri Lanka
aashiqmnm@seu.ac.lk

M.N.A. Hinas

Dept. of Computer Science and Eng.
South Eastern University
Olivil, Sri Lanka
ajmalhinas@seu.ac.lk

Abstract—The implementation of smart crowd management and intelligent infrastructure systems necessitates real-time crowd monitoring to ensure safety, efficiency and responsiveness. Recent developments in deep learning methods and IoT devices have opened new avenues for addressing real-world problems. This study presents a lightweight framework for real-time crowd counting, that utilizes YOLOv8n model for human head detection and the Norfair tracking algorithm for multi-object tracking. The complete system is deployed on a Raspberry Pi 5 edge device connected to a USB camera for video input. Integrating deep learning models on edge devices significantly reduces reliance on high bandwidth data communication and cloud processing. A ThingsBoard-based dashboard provides real-time occupancy visualization and historical crowd data. Experimental evaluations conducted across diverse indoor and outdoor environments demonstrate up to 94% counting accuracy under varying lighting and crowd conditions. The proposed edge-AI solution is scalable, cost-effective and privacy-preserving, as it avoids raw video data transmission for smart crowd monitoring applications.

Keywords—Edge Computing, Internet of Things, Object detection, YOLO, Privacy preserving

I. INTRODUCTION

Rapid urbanization and population growth have increased the need for smart crowd monitoring systems to ensure public safety, use resources efficiently and improve operations in public transportation and other crowded environments [1]–[3]. Traditional methods like manual counting or sensor-based systems, such as infrared or laser scanning, are often expensive, prone to errors in dynamic scenes with occlusions and they do not scale well for use on devices with limited resources like the Raspberry Pi [4]–[6]. Other challenges, including different types of headwear, changing lighting and dense crowds, make it even harder to detect and track people accurately in public

spaces [7], [8]. Using convolutional neural networks (CNNs) to achieve high accuracy and processing speed, deep learning-based object detection models, especially those from the You Only Look Once (YOLO) family, have become reliable solutions for real-time crowd counting [9], [10]. To enhance crowd analysis, recent research has combined YOLO variations with tracking algorithms. For instance, YOLOv8 in conjunction with ByteTrack [1], DeepSORT [3], [4] or centroid tracking [3] has shown encouraging outcomes in public spaces and railway systems. In surveillance applications, compressed YOLOv8 models optimized for edge devices have demonstrated a balance between speed and accuracy [7]. Other initiatives have used hybrid IoT-YOLO systems for density estimation [5], YOLOv5 for open plazas [8] and YOLOv7 with Norfair for multi-object tracking (MOT) [11]. Congestion prediction in transportation systems has been improved by mobile platforms using YOLOv9 [12].

Despite these advancements, a number of restrictions still exist. A lot of methods rely heavily on cloud processing, which adds latency and creates privacy issues. Others find it difficult to generalize in a variety of indoor and outdoor settings, especially when there are differences in illumination and occlusion [2], [9]. Furthermore, their applicability in scalable IoT-driven ecosystems is limited because few studies have focused on real-time data visualization for remote monitoring [1], [5].

This research addresses these limitations by introducing a lightweight, edge deployable framework for real-time crowd counting using the YOLOv8n model integrated with the Norfair multi-object tracking (MOT) algorithm [11]. The proposed system is implemented on a Raspberry Pi 5, enabling efficient on-device inference while reducing latency and eliminating dependence on cloud processing. To support remote moni-

*Corresponding Authors

toring and decision-making, the system also incorporates the ThingsBoard IoT platform, which provides real-time dashboards for visualizing occupancy trends, density levels and historical crowd patterns [5], [12].

The main contributions of this work can be listed as following:

- Development of a YOLOv8n model for optimized for single board edge devices, where reliability and accuracy is expected with limited computational resources.
- Norfair Integration ensures reliable tracking under occlusion and improves real-time head counting.
- Integration with the ThingsBoard IoT platform for remote monitoring, scalability and device management.

The proposed framework shows the feasibility to develop a reliable AI enabled crowd monitoring system with edge processing that reduces latency while preserving data privacy under limited computing resources [2], [7], [9].

The remainder of this paper is organized as follows: Section II reviews related work, Section III details the methodology, Section IV presents results, Section V discusses limitations and future works and Section VI concludes the paper.

II. LITERATURE REVIEW

The developments in deep learning, particularly convolution neural network (CNN) based models, have enabled to approach the challenges of real-time crowd counting with accuracy and automation even in complex dynamic environments [13]. Meanwhile, the You Only Look Once (YOLO) family has emerged as a prominent solution for crowd counting, owing to its real-time performance and robustness [9], [10].

According to recent researches, the YOLO family is a potent solution for real-time detection and counting tasks. While Kalita et al. [10] assessed several deep learning models for both indoor and outdoor people counting, Nguyen et al. [9] used YOLO for human detection on edge devices. Improved detection in dense environments was reported by Kadam et al. [6] after comparing YOLOv8 with different tracking algorithms. For effective human head detection, Goyal et al. [7] presented compressed YOLOv8 architectures, which showed lower computational demand without compromising accuracy. Similar to this, Zhang et al. [8] demonstrated the potential of head-based detection for crowd estimation by proposing an enhanced YOLOv5 variant for real-time head counting in open plazas.

Crowd counting stability has been further improved by integrating tracking algorithms with YOLO. While Alano et al. [4] and Kadam et al. [6] employed DeepSORT for in/out crowd flow estimation, Cruz Antony et al. [1] combined YOLOv8 with ByteTrack for dependable multi-object tracking. Banu et al. [11] used YOLOv7 with Norfair for lightweight, reliable tracking in dynamic scenes, while A. S. et al. [3] used centroid and DeepSORT algorithms in railway systems to enhance passenger counting. Although most systems still rely on full-body detection, which limits precision in dense settings, these studies show that combining YOLO with effective trackers helps maintain count consistency even under occlusion.

Deploying deep learning models on edge devices and integrating them with IoT platforms for real-time monitoring has been the focus of several parallel projects. YOLOv8 deployment on a Raspberry Pi for on-device person detection was demonstrated by Kumar et al. [2], and an IoT–OpenCV–YOLO system for people density estimation with dashboard visualization was created by Jayasingh et al. [5]. Kavanov et al. [12] expanded the method for congestion prediction to mobile platforms. Even though combining IoT and lightweight vision models provide real-time data access, issues such as latency and reliance on the cloud for data processing still pose significant challenges.

Despite recent advancements, the integration of head detection, object tracking and IoT enabled real-time monitoring, into a unified framework for edge deployment remains under-explored. Many existing works either compromise accuracy for efficiency or lack real-time visualization. The work presented on this paper addresses these issues by merging YOLOv8n with Norfair tracking on a Raspberry Pi 5, along with IoT cloud dashboard for live remote monitoring and data visualization. The novel approach of edge processing of crowd video and transmission of only numerical data to cloud ensures, low latency and privacy-aware crowd monitoring, making it suitable for smart public spaces and real-time decision-making.

III. METHODOLOGY

The proposed framework is a lightweight, edge deployable system for real-time crowd counting and density estimation. The system has integrated YOLOv8n for human head detection, Norfair for human tracking, Raspberry Pi 5 for edge deployment and real-time data visualization using ThingsBoard IoT platform. Fig. 1 depicts the complete pipeline of the developed system.

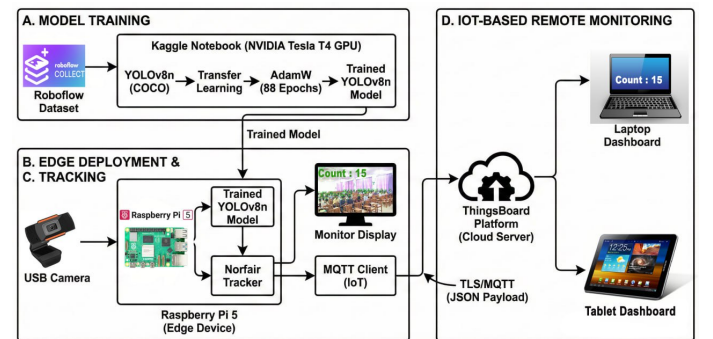


Fig. 1. Complete pipeline: (A) model training, (B) edge deployment, (C) tracking integration and (D) IoT-based monitoring

A. Model Training

The training dataset was taken from the publicly available human head recognition datasets in Roboflow Universe and includes 10,702 annotated images (9,369 for training, 885 for validation, 448 for testing) with single-class YOLO bounding boxes (“head”) [14]. This diversified dataset encompasses both indoor and outdoor settings, variable human densities and numerous views to provide robust generalization.

The YOLOv8n model was selected due to its lightweight architecture (3.2M parameters, 8.7 GFLOPs), which optimizes for efficient inference on edge devices with limited resources [2]. Training of the YOLOv8n model was performed using the AdamW optimizer with a cosine learning rate scheduling strategy, employing a batch size of 32 and a maximum of 150 epochs. The training was conducted on a Kaggle Notebook utilizing an NVIDIA Tesla T4 GPU. Transfer learning was completed with COCO-pretrained YOLOv8n weights. Validation loss and performance indicators were closely monitored to ensure a steady training process. The model learns by minimizing a composite loss function as shown in (1).

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dff}}\mathcal{L}_{\text{dff}} \quad (1)$$

Where:

- $\mathcal{L}_{\text{total}}$: The total training loss used to optimize the object detection model.
- $\mathcal{L}_{\text{box}}$: The bounding box regression loss, which uses measures like CIoU to improve localization accuracy.
- $\mathcal{L}_{\text{cls}}$: Classification loss based on Binary Cross Entropy for object class prediction.
- $\mathcal{L}_{\text{dff}}$: The Distribution Focal Loss improves the accuracy of bounding box distribution predictions.
- λ_{box} , λ_{cls} and λ_{dff} are its own weighting coefficients for each loss component.

B. Edge Deployment

The Raspberry Pi 5 (4GB RAM, 2.4 GHz Broadcom BCM2712 quad-core Cortex-A76 CPU) was used as the inference platform. A 720p USB camera was employed as the video input device. Being a lightweight architecture, YOLOv8n model allowed for efficient inference on the Raspberry Pi 5 platform. The software stack was built on Raspberry Pi OS (64-bit) with Python 3.9, incorporating Ultralytics YOLO, OpenCV, NumPy and Paho-MQTT as essential libraries. A custom real-time inference pipeline managed the full processing chain from frame acquisition and head detection through object tracking and headcount aggregation to IoT data transmission via MQTT. The processing workflow is outlined in the following steps:

- Capturing video frames from the connected camera.
- Applying the trained YOLOv8n model to each frame for detection.
- Generating bounding boxes around human heads.
- Displaying the dynamic crowd count (on the connected display unit).

Several optimizations were applied, including FP16 quantization, multi-threaded video capture and passive cooling, to ensure stable performance during prolonged operation [2], [9].

C. Tracking Integration

To maintain consistency across consecutive detection frames, the YOLOv8n model was integrated with the Norfair tracking algorithm [11]. By using a combined motion and geometry-based distance metric, the tracker can reliably follow objects across frames even when there is occlusion or dense

crowd movement. This helps maintain consistent tracking over time and improves the stability of real-time head counting. The Norfair tracker calculates a composite distance metric as shown in (2), which balances motion prediction and geometric consistency.

$$d(i, j) = \lambda \cdot \underbrace{\|\mathbf{x}_i - \hat{\mathbf{x}}_j\|_{\Sigma_j}}_{\text{Mahalanobis (motion)}} + (1 - \lambda) \cdot \underbrace{(1 - \text{IoU}(b_i, \hat{b}_j))}_{\text{Geometric}} \quad (2)$$

where $d(i, j)$ denotes the association cost between detection i and track j ; $\mathbf{x}_i$ and $\hat{\mathbf{x}}_j$ represent the detected and predicted states, respectively; Σ_j is the covariance matrix; b_i and $\hat{b}_j$ are the corresponding bounding boxes; IoU measures the spatial overlap between bounding boxes.

$\lambda = 0.7$ is a tunable weight slightly favoring motion. This value was selected based on experiments to reduce ID switches while keeping tracking stable in typical crowded situations [11].

The system processes the input video at a resolution of 1280 × 720 pixels (720p) and applies a scaling factor of 0.4 to improve computing efficiency on edge devices. For detection, YOLOv8n uses a non-maximum suppression IoU threshold of 0.4 to remove overlapping boxes and a confidence threshold of 0.2 to detect voids. In Norfair, a maximum hit count of 10 frames, a track start delay of 3 frames and a matching distance threshold of 50 pixels are used to maintain object tracking during short-term blockages.

Each detected head is assigned a unique ID, which is removed once the object leaves the scene. The final count is calculated by monitoring active IDs at short intervals, helping to avoid double counting. The combination of YOLOv8n and Norfair provides accurate and consistent real-time tracking.

D. IoT-Based Monitoring and Privacy

IoT integration was achieved via the ThingsBoard platform, with MQTT employed as a lightweight communication protocol. Since the proposed framework utilizes edge-analysis, simple JSON payloads are sufficient for sending numerical results to the ThingsBoard IoT cloud. The structured JSON payloads include timestamp, device ID, head count, detection confidence and system status. No personally identifiable information including facial features, screenshots or video recordings is shared or stored.

Unlike conventional cloud-based approaches that require continuous transmission of raw video streams to remote servers for processing, this edge-processing architecture performs detection and tracking locally on the edge device and sends only aggregated crowd statistics to the cloud platform, significantly reducing network bandwidth usage, lowering system latency and enhancing privacy by preventing raw video from leaving the device.

The native TLS encryption and token-based authentication mechanisms of the IoT cloud platform ensure data security. The dashboard visualizes real-time crowd data and historical headcount trends. The monitoring dashboard is accessible only to authorized users, allowing real-time crowd monitoring while reducing privacy risks.

E. Testing and Validation

The trained system was deployed and evaluated in diverse real-world environments with real-time videos, including lecture halls, classrooms, auditoriums and crowded events, to verify robustness under varying lighting conditions, crowd densities and occlusion.

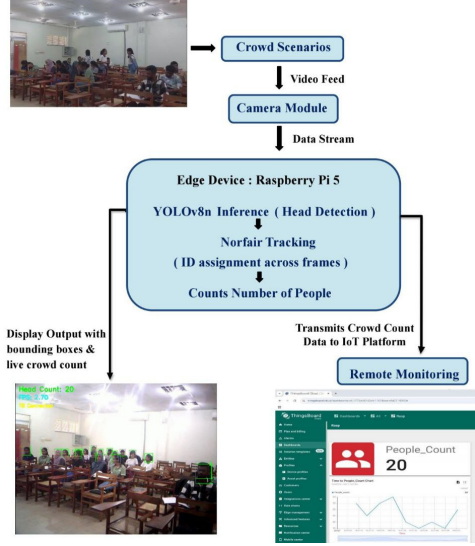


Fig. 2. System Architecture of Method

The Fig. 2 shows the overall system architecture indicating all critical stages from training to real-time edge deployment and real-time monitoring on IoT dashboard.

IV. RESULTS AND DISCUSSION

Several assessments were done on the developed system, such as testing of model accuracy, crowd counting reliability, multi-object tracking, real-time inference speed on the Raspberry Pi 5 and IoT dashboard functionality. Metrics such as precision, recall, F1-score, FPS (frames per second), counting errors and mAP@0.5 (mean Average Precision at IoU threshold 0.5) were used to evaluate the system. Also the system was evaluated for bounding box accuracy, system stability and the dashboard's usability with live video feeds from different real-world environments through qualitative testing.

A. Model Training Performance Evaluation

The behavior of the loss components during training are shown in Fig. 3(a), Fig. 3(d), Fig. 3(e). As illustrated they dropped quickly at beginning but eventually continued to improve. At the same time, performance metrics like Precision and Recall, as shown in Fig. 3(c) improved, indicating the model improving in detecting objects. A high-performance state was quickly reached, with the mAP@0.5 as shown in Fig. 3(b), stabilizing near an excellent 0.95. The more stringent mAP@0.5-0.95 metric also stabilized near 0.50. After approximately 88 epochs, the model was stopped due to the clear signs of convergence and the onset of overfitting. Specifically, from

approximately 60 epochs onward, the Validation Loss curves began to stabilize and showed a slight upward drift, while the Training Loss continued its slow decrease.

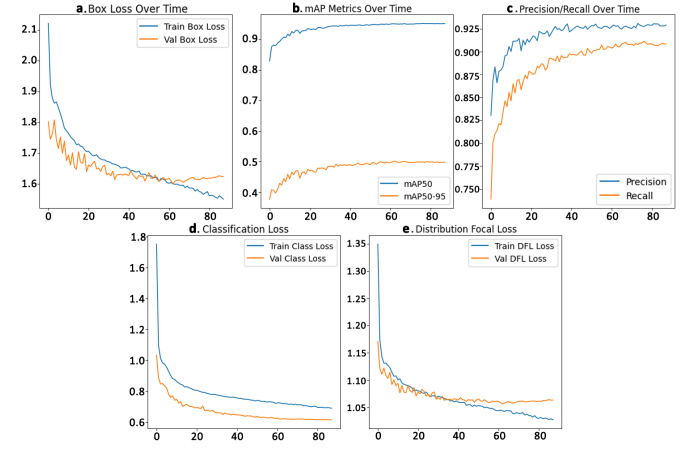


Fig. 3. Training and Validation Metrics for Object Detection Model (a) Box Loss Over Time, (b) mAP Metrics Over Time, (c) Precision/Recall Over Time, (d) Classification Loss and (e) Distribution Focal Loss

B. Tracker Comparison and Ablation Study

To evaluate the effectiveness of different tracking strategies, an ablation study was conducted by integrating YOLOv8n with several multi-object tracking algorithms.



Fig. 4. Qualitative comparison of different tracking methods on a sample crowd scene: (A) YOLOv8n, (B) YOLOv8n + DeepSORT, (C) YOLOv8n + Norfair and (D) YOLOv8n + ByteTrack.

Fig. 4 presents a qualitative comparison of tracking approaches on a representative crowd scene, showing improved object association and counting stability when tracking is applied.

The accuracy values in Table I were calculated by comparing the predicted count with the actual crowd count using the following equation (3):

$$Accuracy(\%) = \left(1 - \frac{|Actual - Predicted|}{Actual\ Count}\right) \times 100 \quad (3)$$

TABLE I
PERFORMANCE COMPARISON OF YOLOv8n WITH DIFFERENT TRACKING METHODS

Method	Accuracy (%)	FPS	Computational Load	Suitability for Edge
YOLOv8n	77.8	4-5	Low	Moderate
YOLOv8n + DeepSORT	98.4	0.8-1.5	High	Low
YOLOv8n + ByteTrack	93.7	1.5-2.2	Medium	Moderate
YOLOv8n + Norfair	96.8	2-3	Low	High

Table I summarizes the quantitative comparison of the evaluated tracking approaches. Considering the trade-off between accuracy, processing speed and computational load, YOLOv8n combined with Norfair was selected for the proposed system. This configuration provides reliable tracking performance while maintaining efficient inference suitable for real-time edge deployment.

C. Head Detection and Tracking Accuracy

The performance of the YOLOv8n model for head detection was quantitatively evaluated using metrics such as mAP@0.5, precision, recall and F1-score using (3), (4), (5) respectively [1],[7],[11].

- **Precision (P):** Evaluates positive prediction accuracy.

$$P = \frac{TP}{TP + FP} \quad (4)$$

- **Recall (R):** Measures the model's ability to find all relevant cases.

$$R = \frac{TP}{TP + FN} \quad (5)$$

- **F1-Score:** The harmonic mean of precision and recall.

$$F1 = 2 \times \frac{P \times R}{P + R} \quad (6)$$

Where: TP (True Positives), FP (False Positives) and FN (False Negatives).

The developed model achieved an mAP@0.5 of 0.9520, an mAP@0.5-0.95 of 0.4990, a precision of 0.9290 and a recall of 0.9083. The computed F1-score of 0.9185 indicates balanced performance. The qualitative results demonstrate head detection and stable tracking and the system successfully maintains unique IDs across frames. This demonstrates its ability to preserve temporal consistency.

D. Edge Deployment and Real-Time Inference Speed

During real-time testing on the Raspberry Pi 5 platform, the system achieved an average inference rate of 2–3 frames per second (FPS). Although this FPS may seem low for highly dynamic scenarios, it is consistent with prior works; for example, [9] reports a YOLO-based human recognition system running on a Raspberry Pi 3B at 1.96 FPS. The slightly higher FPS in our case is mainly due to optimizations in the model and tracking pipeline. For typical crowd movement speeds, this frame rate is sufficient to maintain reliable detection

and counting. For faster-moving crowds or higher real-time requirements, FPS could be further improved by deploying more powerful edge devices.

The performance confirms feasibility of YOLOv8n for edge deployment and meets the requirements for crowd monitoring applications. As discussed in Section B of the Methodology, resource usage (CPU, RAM) remained within acceptable limits during extended use.

E. IoT Dashboard Results

The data generated from edge-analysis was transmitted to the IoT cloud platform using MQTT protocol. The IoT dashboard provides a user-friendly monitoring interface displaying both historical trends and current occupancy metrics over time. This integration allowed for remote observation of crowd dynamics.

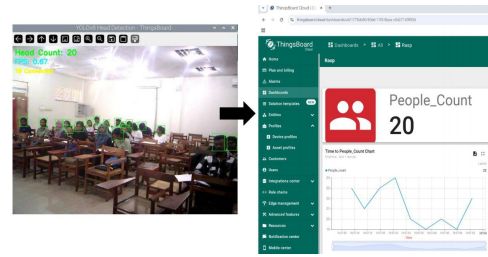


Fig. 5. ThingsBoard Monitoring Interface for Live Crowd Data

F. Evaluation Across Lighting and Camera Angle Variations

The robustness of the proposed system was first evaluated in a high-density lecture hall environment under controlled variations of illumination and camera viewpoint. Three camera positions - front, side-top and rear views, were tested to find the optimal viewpoint for minimizing occlusion. Also, two illumination conditions- well-lit and low-light were tested to evaluate the model's tolerance to brightness variations.

a) Testing Scenarios - Lighting Variation



Fig. 6. Crowd detection under different lighting conditions: (a) well-lit and (b) low-light

Fig. 6 and 7 show sample detection frames from the test cases. With clear bounding boxes even under low light conditions, visual results show that rear views can sometimes lead to under-counting due to limited head visibility, while side-top views provide a wider field of view and reduce overlap between individuals.

b) Testing Scenarios - Camera Angle Variation

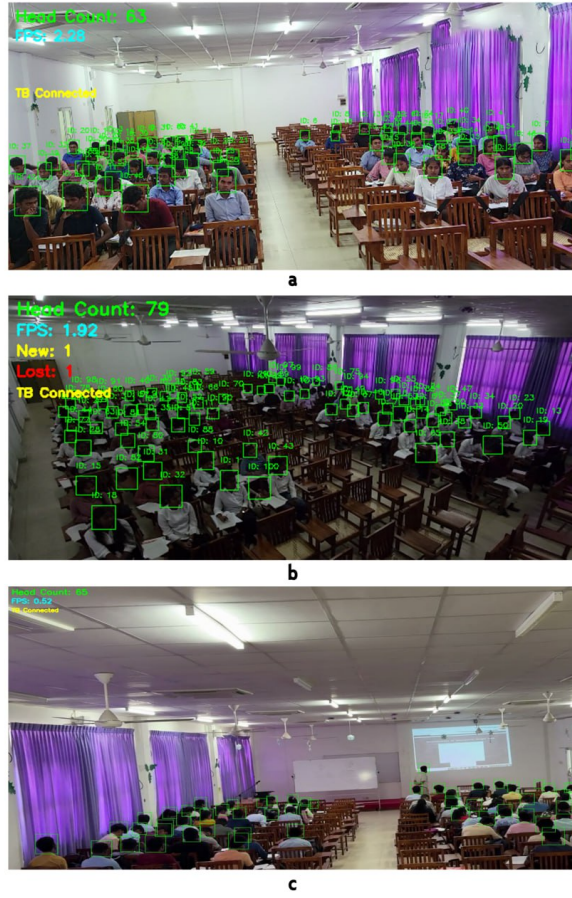


Fig. 7. Real-time head detection from different camera viewpoints: (a) Front view, (b) Side high-angle view and (c) Back view

TABLE II
CROWD COUNTING PERFORMANCE UNDER DIFFERENT VIEWS AND LIGHTING

Condition	Actual Count	Predicted Count	Absolute Error ($\pm$)	Accuracy (%)
Front View	69	63	6	91.3
Side-Top	81	79	2	97.5
Back View	69	65	4	94.2
Well-lit	58	54	4	93.1
Low light	70	65	5	92.85

Fig. 6 and 7, along with Table II, show that the developed system maintains consistent accuracy under different lighting conditions and viewing angles. The results demonstrate reliable detection and robustness across these variations. They also show that the best performance is achieved with a side-top camera view, where there are fewer obstructions and people are more clearly visible.

G. Evaluation Across Different Real-World Locations

This system was tested in eight real-world scenarios (Fig. 8, Table II), using multiple videos at each location to verify its consistency and robustness.

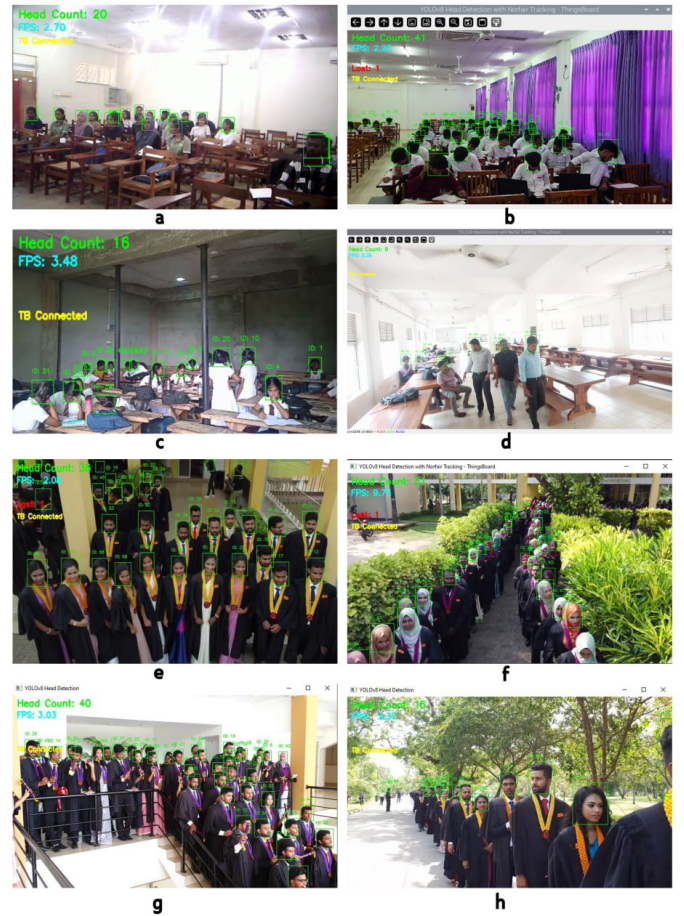


Fig. 8. Representative scenes of crowd counting across different locations and crowd densities. (a) Low-density lecture hall, (b) Medium-density classroom, (c) Educational center, (d) Indoor hall, (e) Convocation group, (f) Outdoor queue, (g) Graduation gathering, (h) Outdoor pathway

TABLE III
CROWD COUNTING PERFORMANCE FROM DIFFERENT REAL-WORLD LOCATIONS

Location / Scenario	Actual Count	Predicted Count	Abs. Error	MAPE (%)	Accuracy (%)
Low-density lecture hall (Fig. 8a)	21	20	1	4.76	95.24
Medium-density hall (Fig. 8b)	42	41	1	2.38	97.62
Educational center (Fig. 8c)	17	16	1	5.88	94.12
Indoor hall environment (Fig. 8d)	10	10	0	0.00	100.00
Convocation group (Fig. 8e)	38	36	2	5.26	94.74
Outdoor queue (Fig. 8f)	50	48	2	4.00	96.00
Graduation gathering (Fig. 8g)	41	40	1	2.44	97.56
Outdoor pathway (Fig. 8h)	17	16	1	5.88	94.12
Average	-	-	1.13	3.83	96.18

The Mean Absolute Percentage Error (MAPE) for each location was calculated using the predicted and actual crowd

counts as follows (7) :

$$MAPE (\%) = \frac{|Actual Count - Predicted Count|}{Actual Count} \times 100 \quad (7)$$

The real-world results showed that the developed system was very adaptable to various lighting conditions, crowd sizes and viewing angles, achieving up-to 94% accurate count across all sites as shown in Fig. 8 and table III. The MAPE values remain low, indicating that deviations from the actual count are minimal. Data was successfully transmitted to the IoT dashboard in all cases, indicating reliable connectivity as well.

YOLOv8n and Norfair effectively reduced duplicates by ensuring consistent IDs and steady detection. The model's lightweight design allowed for continuous inference and real-time updates to the IoT dashboard with acceptable few seconds of latency, despite the Raspberry Pi 5's limited processing power compared to GPU systems.

TABLE IV
COMPARISON OF PREVIOUS APPROACHES WITH THE PROPOSED SYSTEM

Ref	Approach	Platform / Deployment	Real-Time Capability	Accuracy / Performance
[1]	YOLOv8+ ByteTrack	General CPU/GPU	Real-time	86.6%(Accuracy), F1=0.729
[3]	YOLOv8 + DeepSORT	Smart Railway System	Real-time	92.3%
[6]	YOLOv8+ Multiple Tracking	Video pipeline	Real-time	BoT-SORT (Best Accuracy: 79.45%)
[11]	YOLOv7+ Norfair	No custom hardware specified	Real-time	MOTA=80.2% (MOT17), 79.7% (MOT20)
Proposed Work	YOLOv8n+ Norfair	Raspberry Pi 5 / Edge	2-3 FPS Real-time	> 94%

Table IV compares the proposed system with several crowd counting approaches. Prior works often rely on heavier trackers or GPU platforms, with limited reporting of FPS or quantified accuracy. In contrast, our system balances lightweight design, edge deployment, real-time feasibility and high accuracy, demonstrating a practical trade-off suitable for IoT edge devices.

Future work will focus on expanding the dataset, performing comparative analyses and optimizing the model through pruning and quantization. Deployment on GPU-supported edge devices will improve frame rates and enhancements to the IoT dashboard will increase usability for real-time monitoring. Additionally, We plan to address privacy concerns by developing a simple threat model to ensure that only aggregate data is transmitted and identifiable information is protected.

V. CONCLUSION

This paper presented a real-time crowd counting system that combines YOLOv8n for head detection, Norfair for object tracking and Things Board IoT for remote monitoring. The model was trained on a head-detection dataset and deployed on a Raspberry Pi 5 to achieve an efficient edge-based solution without dependence on cloud resources. Experiments in

various lighting situations, camera angles and real-world locations showed great detection accuracy and constant tracking performance.

The solution achieved over 94% accuracy in all tested scenarios and delivered live IoT dashboard updates with minimal latency. The side-top camera position offered optimum coverage, minimized obstruction and less occlusion. Overall, the proposed system demonstrated the feasibility of deployment of lightweight deep learning models for simple crowd monitoring applications on edge devices through edge analytics.

REFERENCES

- [1] J. Cruz Antony, et al., "Advancing Crowd Management through Innovative Surveillance using YOLOv8 and ByteTrack," in *Proc. 8th Int. Artif. Intell. Data Process. Symp. (IDAP)*, Malatya, Turkey, 2024, pp. 1–6, doi: 10.1109/IDAP64064.2024.10710788.
- [2] P. Kumar, et al., "Pi-Vision: Leveraging YOLO V8 for Real-Time Person Detection and Recognition on Raspberry Pi," in *Proc. 5th Int. Conf. Data Intell. Cogn. Informatics (ICDICI)*, 2024, pp. 1446–1450, doi: 10.1109/ICDICI62993.2024.10810836.
- [3] A. S., et al., "Exploring the Integration of Centroid and Deep Sort Algorithms with YOLOv8 Detector for Passenger Counting in Smart Railway Systems," in *Proc. 2nd Int. Conf. Netw., Embedded Wireless Syst. (ICNEWS)*, 2024, pp. 1–8, doi: 10.1109/ICNEWS60873.2024.10731079.
- [4] J. M. L. Alano, et al., "Real-Time In/Out Crowd Counting System Utilizing YOLOv8 and DeepSORT," in *Proc. 8th Int. Conf. Electr., Telecommun. Comput. Eng. (ELTICOM)*, 2024, pp. 1–6, doi: 10.1109/ELTICOM64085.2024.10864842.
- [5] S. K. Jayasingh, et al., "Integrated Crowd Counting System Utilizing IoT Sensors, OpenCV and YOLO Models for Accurate People Density Estimation in Real-Time Environments," in *Proc. 1st Int. Conf. Cogn., Green Ubiquitous Comput. (IC-CGU)*, 2024, pp. 1–6, doi: 10.1109/IC-CGU58078.2024.10530804.
- [6] S. S. Kadam, et al., "Crowd counting using YOLOv8 and various tracking algorithms," in *Proc. OPJU Int. Technol. Conf. (OTCON) Smart Comput. Innovation Advancement Ind. 4.0*, 2024, pp. 1–9, doi: 10.1109/OTCON60325.2024.10688023.
- [7] H. Goyal, et al., "Compressed YOLOv8 Models for Efficient Human Head Detection in Transit Video Surveillance," in *Proc. IEEE Int. Conf. Intell. Signal Process. Effective Commun. Technol. (INSPECT)*, 2024, pp. 1–6, doi: 10.1109/INSPECT63485.2024.10896114.
- [8] H. Zhang, et al., "Dynamic Video Streaming Real-time Headcount Based on Improved YOLO V5 for Open Plazas," in *Proc. 7th Int. Conf. Commun., Image Signal Process. (CCISP)*, 2022, pp. 233–237, doi: 10.1109/CCISP55629.2022.9974407.
- [9] H. H. Nguyen, et al., "YOLO Based Real-Time Human Detection for Smart Video Surveillance at the Edge," in *Proc. 8th Int. Conf. Commun. Electron. (ICCE)*, 2021, pp. 439–444, doi: 10.1109/ICCE48956.2021.9352144.
- [10] D. Kalita, et al., "Vision-Based People Counting System for Indoor and Outdoor Environments using different Deep Learning Models," in *Proc. IEEE Int. Conf. Comput. Vis. Mach. Intell. (CVMI)*, 2024, pp. 1–6, doi: 10.1109/CVMI61877.2024.10782123.
- [11] R. Banu, et al., "Multiple Object Tracking using YOLOv7 Based Object Detection and Norfair Tracking," *Tuijin Jishu/J. Propulsion Technol.*, vol. 44, no. 6, pp. 3107–3114, 2023, DOI: <https://doi.org/10.52783/tjjpt.v44.i6.3862>.
- [12] A. Kavanoz, et al., "Deep Learning Based Mobile Platform for Predicting Passenger Congestion in Public Transportation," in *Proc. 8th Int. Artif. Intell. Data Process. Symp. (IDAP)*, Malatya, Turkey, 2024, pp. 1–6, doi: 10.1109/IDAP64064.2024.10710788.
- [13] S. V. Vasantha et al., "Face and Object Detection Algorithms for People Counting Applications," in *Proc. 2nd Int. Conf. Autom., Comput. Renew. Syst. (ICACRS)*, IEEE, 2023, pp. 1188–1193, doi: 10.1109/ICACRS58579.2023.10405114.
- [14] Head Datasets – OCIDU, "Head Datasets," Roboflow Universe. [Online]. Available: <https://universe.roboflow.com/head-ocidu/head-datasets-9gucc/dataset/1>